Optimizing Llama.cpp for Heterogeneous AMD GPUs

https://gemini.google.com/app/71f1f972e7c2ce89

User prompt: Based on these stats, which inference English be would be better than my current llama.cpp setup for my iq3 Qwen 3.8 flash next build? 2 constraints I have is using 2 and gpu's with older architecture (mi5016gb and rx6950xt), as well as using multiple parallel agents (2 right now at q4 context but I was hoping for more)

Response: **None of the high-ranking engines from that benchmark will work for your build; sticking with and tuning `llama.cpp` remains your best—and essentially only viable—path.**

While the benchmark shows engines like Basalt, Infernix, and capy-flash outperforming `llama.cpp` by 4x to 9x, those numbers are strictly tied to hardware and workload configurations that directly conflict with your setup.

Why the Benchmark Engines Fail Your Constraints

1. Hardware Mismatch: Heterogeneous AMD GPUs

- **NVIDIA Blackwell Architecture Lock-in:** The top performers in the post (**Basalt** and **Infernix**) are CUDA-only engines custom-built specifically for NVIDIA Blackwell (`sm_120`, RTX 50-series) to leverage native NVFP4 tensor cores. They do not support AMD ROCm.
- **Dual-GPU Engines Require Identical NVIDIA Silicon:** While the author notes that **capy-flash** is "built mostly for two GPUs", it was designed for two matching NVIDIA cards using CUDA peer-to-peer tensor parallelism. It cannot run on AMD hardware.
- **Mixed ROCm Targets (`gfx906` vs. `gfx1030`):** Your two GPUs belong to completely different AMD hardware generations:
  - **MI50 16GB:** GCN 5.1 / CDNA precursor architecture (`gfx906`, Wavefront 64).
  - **RX 6950 XT 16GB:** RDNA 2 architecture (`gfx1030`, Wavefront 32). Even **Strata** (which has experimental AMD support for single RDNA2/3 cards) does not support heterogeneous ROCm execution across mismatched AMD generations. Compiling and running unified kernels across both architectures causes runtime crashes in specialized engines.

2. The Parallel Agent Bottleneck

- **MoE Expert Cache Thrashing:** The non-`llama.cpp` engines in the benchmark achieve their high speeds via **dynamic VRAM expert caching** (keeping a few hot experts in VRAM and fetching misses from system RAM). This design works well for a **single user stream**. When running multiple parallel agents, each agent queries different experts simultaneously, collapsing the cache hit rate and causing severe PCIe/host-RAM bottlenecks.
- **Concurrency Limitations:** Specialized engines like Strata and Basalt are primarily architected for single-request interactive sessions. Serving multiple parallel agents requires robust continuous batching and isolated context slot allocation—features where `llama.cpp`'s server is mature.

3. Why `llama.cpp` Scored Low in the Test

The benchmark post itself highlights why the `llama.cpp` figure is misleading:

- **MTP Was Disabled:** `llama.cpp` was run with Multi-Token Prediction (MTP) turned off, while the competitors ran custom speculative drafters.
- **Full FP16 KV Cache:** `llama.cpp` was benchmarked using `f16` KV cache, whereas all other engines used `int8` KV cache.
- **NVFP4 Quants:** The test used converted NVIDIA NVFP4 weights, which are not native to `llama.cpp`'s compute pipeline.

Maximizing Your Setup in `llama.cpp`

`llama.cpp` remains uniquely suited for your build because it natively supports heterogeneous layer splitting (`--split-mode layer`) across disparate GPUs and handles your **IQ3** GGUF weights directly.

To scale beyond your current 2 parallel agents and improve throughput:

1. **Quantize the KV Cache:** Run your context cache in 4-bit or 8-bit mode to drastically reduce VRAM usage per agent slot:

   ```
   --ctk q4_0 --ctv q4_0
   ```

   At 32GB total VRAM (16GB + 16GB), freeing up KV memory is the single most effective way to scale from 2 to 4+ concurrent slots (`-np 4`).

2. **Handle Heterogeneous GPU Stability:** If ROCm throws driver or hipBLAS dispatch errors when targeting both `gfx906` and `gfx1030` in the same binary, switch to `llama.cpp`'s **Vulkan backend** (`-DGGML_VULKAN=ON`). Vulkan treats each card as an independent device without ROCm target-matching issues.

3. **Tune Tensor Split for Capacity:** Ensure layer allocations match the actual headroom after KV cache reservation using `--tensor-split` (e.g., `--tensor-split 1,1` or biasing more layers to whichever card isn't running your display output).

4. **Enable Speculative Drafting / MTP:** If you have spare system RAM or host CPU threads, test pairing your IQ3 base model with an MTP draft model (`--draft` or the Qwen 3.8 draft head) to close the decode speed gap without switching runtimes.

User prompt: How are these engines made and why are they locked to cuda? Could I start a project that looks at the comparability and works done between each of these engines and make a vulkan based alternative? How would I start that?

Response: How These Engines Are Built and Why They Are Locked to CUDA

Engines like **Basalt**, **Infernix**, **ninfer**, and **Strata** were built to solve a specific problem: **running massive Mixture-of-Experts (MoE) models on consumer/prosumer workstations where the weights exceed available VRAM**.

1. Core Architecture Pattern

Instead of holding all experts in VRAM, these engines split execution:

- **Permanent Attention Core:** The model's non-MoE weights (embeddings, self-attention / QSA, linear projections, norm heads, and router) remain permanently resident in VRAM.
- **Streaming Expert Cache:** The routed experts live in system RAM. When the router picks top-k experts for a token, a cache manager checks if they are in a dynamic VRAM pool. Hits execute immediately; misses are streamed across PCIe via direct memory access (DMA).
- **Integrated Speculative Decoding:** They embed Multi-Token Prediction (MTP) or speculative heads (like DFlash2) directly into the execution graph, generating proposals and verifying them in grouped batches.

2. Why They Are Locked to CUDA

- **Hardcoded PTX / Inline Assembly:** To maximize decode throughput, these runtimes bypass generic compute frameworks. They use inline PTX assembly for hardware-level instructions like `mma.sync` or Blackwell-exclusive `m16n8k32` operations. These assembly instructions physically do not exist outside NVIDIA hardware.
- **NVIDIA-Specific Quantization (NVFP4):** Native NVFP4 requires Blackwell (`sm_120`) tensor cores. AMD hardware (CDNA/RDNA) lacks NVFP4 hardware dequantizers and tensor layouts.
- **CUDA Streams and Pinned DMA:** Overlapping host-to-device memory copies with compute requires zero-overhead stream orchestration (`cudaMemcpyAsync`, pinned host buffers via `cudaHostAlloc`, and CUDA stream callbacks). The authors designed their memory allocators entirely around NVIDIA's Unified Virtual Addressing (UVA).

Comparative Breakdown of the Landscape

Engine	Upstream Heritage	Core Strategy	Quant Formats Supported	Hardware Target
Basalt	Fork of Strata	Pinned RAM streaming + peer-GPU tier + MTP drafter	NVFP4, GSQ-RCO (IQ3, Q2), Unsloth UD	Single/Dual NVIDIA Blackwell (`sm_120`)
Infernix	Fork of NInfer	Asynchronous prefetch ring + host/device split compute	Dense8 FP8, NVFP4, GSQ-RCO, Q8_K	Single NVIDIA RTX 50-series
Strata	Original base runtime	Route-tail skipping + multi-worker host thread pool	GGUF (UD quants, IQ4_NL, Q8_0)	NVIDIA CUDA (experimental single RDNA)
llama.cpp	Independent	Full static layer offloading, standard sequential compute	All GGUF types (IQ1–IQ4, Q8_0, etc.)	Cross-platform (CUDA, ROCm, Vulkan, Metal)

Feasibility of a Vulkan-Based Alternative

Creating a Vulkan-based MoE streaming engine is technically viable. In fact, **Vulkan is superior to ROCm for your specific hardware combination** because ROCm struggles to compile single binaries that dispatch cleanly across mismatched AMD targets (`gfx906` on the MI50 and `gfx1030` on the RX 6950 XT). Vulkan abstracts both GPUs as standard `VkPhysicalDevice` objects regardless of underlying microarchitecture.

Here is how CUDA concepts translate directly to Vulkan:

- **Async PCIe DMA Transfers:** Use dedicated **Transfer Queue Families** (`VK_QUEUE_TRANSFER_BIT`) separate from Compute Queue Families. Overlap compute shaders with background PCIe transfers using `vkCmdCopyBuffer` from host-visible, coherent staging buffers.
- **Synchronization:** Replace CUDA Events and Streams with **Vulkan Timeline Semaphores** (`VK_KHR_timeline_semaphore`). This allows the host scheduler to signal and wait on individual expert streaming stages fine-grained to the microsecond.
- **Matrix Acceleration:** Replace proprietary CUDA MMA intrinsics with `VK_KHR_cooperative_matrix` (supported on modern AMD drivers). This allows SPIR-V shaders to hit hardware matrix units (Matrix Cores on the MI50 and Wave Matrix Multiply-Accumulate / WMMA on RDNA).
- **Multi-GPU Orchestration:** You instantiate two `VkDevice` handles. You can use `VK_KHR_external_memory` to share buffers or handle cross-GPU routing through the host.

Project Blueprint: How to Build It

Phase 1: Dissect the Existing Engines

- Clone **Strata** and **Infernix**. Because Basalt and Infernix forked from earlier codebases, examining their commit histories will show you the exact delta where memory streaming was added to a standard transformer loop.
- Isolate the scheduler logic: Look specifically for their **route prediction** and **prefetch ring** implementations. (Infernix predicts the next layer's router output to begin PCIe transfers one layer early).
- Inspect `ggml-vulkan.cpp` inside `llama.cpp`. This gives you tested, open-source SPIR-V shader templates for dequantizing GGUF blocks (like `IQ3_S` and `Q8_0`) into standard FP16 registers.

Phase 2: Build the Vulkan Streaming Harness

Before writing any neural network math, build a lightweight C++ harness to benchmark raw streaming throughput:

1. Initialize your two GPUs as independent `VkDevice` handles.
2. Allocate pinned host memory via `vkAllocateMemory` with `VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT`.
3. Create a ring buffer on each GPU for dynamic expert slots.
4. Stress-test asynchronous DMA copies using timeline semaphores while a dummy compute shader runs simultaneously on the compute queue. Measure PCIe bandwidth to confirm you achieve near link saturation without stalling compute.

Phase 3: Implement Matrix Multiplication Shaders

1. Write SPIR-V compute kernels targeting your weights (GSQ-RCO or IQ3 GGUF layout).
2. For vector-matrix operations (decode phase, batch size = 1 to 4): standard subgroup reduction shaders often outperform cooperative matrix operations because decode is memory-bandwidth bound rather than compute-bound.
3. For batched prefill: compile kernels with `VK_KHR_cooperative_matrix` to leverage the matrix cores on both cards.

Phase 4: Model Graph and Speculative Scheduler

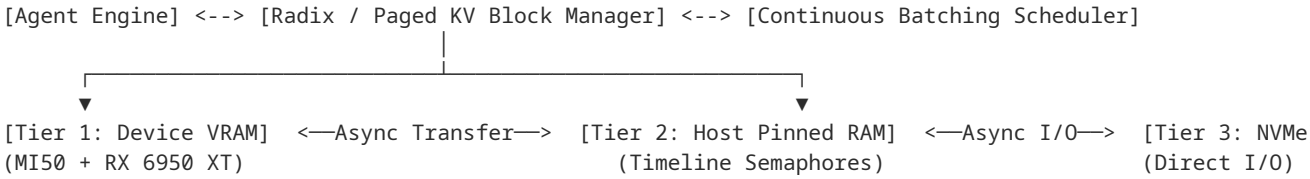
1. **Model Loader:** Write a GGUF parser to extract layer tensors, keeping non-MoE layers in VRAM and expert indices indexed in host RAM.
2. **Drafter / MTP Integration:** Implement the single speculative proposal head. The draft head runs quickly to propose 2–4 tokens, allowing your GPU to evaluate multiple tokens in a single forward pass, reducing the number of round-trip PCIe transfers per generated token.
3. **Agent Slot Manager:** Implement continuous batching for your agents by maintaining separate KV-cache descriptor sets per agent while dispatching batched expert queries to the unified cache.

Which part of the pipeline would you prefer to tackle first: the Vulkan memory-streaming harness or adapting the dequantization compute shaders for your IQ3 weights?

User prompt: I was hoping to get this up in stages - can you specify how it could be optimised for parallel agents workflows, cold storage for kvcaching, vulkan operability and being more model agnostic?

Response: Architectural Foundations

To build an inference runtime optimized for parallel agents, cold-swapped context, and heterogeneous AMD GPUs, four foundational systems must work together:



Pillar 1: Parallel Agent Architecture & Prefix Caching

Agentic workloads behave differently from simple chat requests: they invoke tools, await API responses, spawn sub-tasks, and share large system prompts.

- **Radix-Tree Prefix Sharing:** Agents often share a massive system prompt (tool definitions, schemas, environment rules). Instead of duplicating these tokens across agent contexts, use a **Radix Tree** over the physical KV cache. When Agent B forks from Agent A's prompt, Agent B's page table points to the exact same physical memory blocks with an incremented reference count.
- **Continuous Batching with Agent State Management:** Traditional engines batch at the sequence level; an agent blocked on a tool call stalls the whole batch. Implement **iteration-level scheduling** where each agent is a state machine:

$$\text{State} \in \{\text{Prefill}, \text{Decoding}, \text{WaitingForTool}, \text{SwappedOut}, \text{Complete}\}$$

  At each decoding step, the engine only assembles tokens from active agents, leaving idle slots open.

Pillar 2: Hierarchical Cold Storage for KV Caches

When an agent executes an external tool or waits for user input, keeping its KV cache in high-speed VRAM wastes bandwidth and limits concurrency.

- **Three-Tier Storage Hierarchy:**
  - **L1 (VRAM):** Active KV blocks for decoding tokens right now.
  - **L2 (Pinned Host RAM):** Suspended agent blocks. Staged via host-visible, coherent memory buffers.
  - **L3 (Local NVMe):** Deep sleep / long-term agent sessions saved to disk via direct OS file descriptors (`O_DIRECT` or `io_uring`).
- **Asynchronous Paging Engine:**
  - Partition the KV cache into fixed-size physical blocks (e.g., 16 or 32 tokens per block).
  - When an agent enters `WaitingForTool`, its block table is tagged as cold. A background thread dispatches non-blocking PCIe DMA transfers to evict those blocks from L1 to L2.
  - When the tool returns, a **Prefetch Hint** triggers a transfer back to L1 before the agent's next decode tick is scheduled.

Pillar 3: Heterogeneous Vulkan Operability

Your hardware consists of two fundamentally distinct AMD microarchitectures:

- **AMD Radeon Instinct MI50:** GCN 5.1 / CDNA precursor (`gfx906`), 1024 GB/s HBM2 bandwidth, native Wavefront 64.
- **AMD Radeon RX 6950 XT:** RDNA 2 (`gfx1030`), 576 GB/s GDDR6 + Infinity Cache, default Wavefront 32.

To run both seamlessly under Vulkan without ROCm target-mismatch crashes:

- **Dual `VkDevice` Abstraction:** Initialize each GPU as a separate, isolated logical device. Run layer-pipelined model execution: GPU 0 executes layers $0 \dots M$, passes the hidden state buffer to GPU 1 via host-pinned staging or `VK_KHR_external_memory`, which finishes layers $M+1 \dots N$.
- **SPIR-V Specialization Constants for Subgroups:** Do not hardcode wave sizes in compute shaders. Declare subgroup sizing dynamically:

  ```
  layout(constant_id = 0) const uint SUBGROUP_SIZE = 64; // Set to 64 on MI50, 32 on RX 6950 XT
  ```

- **Dedicated Queue Decoupling:** Use two distinct queues per device:
  1. `VK_QUEUE_COMPUTE_BIT` for matrix multiplication and attention shaders.
  2. `VK_QUEUE_TRANSFER_BIT` for background KV cache paging and expert weight streaming. Synchronize compute and transfer steps without CPU roundtrips using `VK_KHR_timeline_semaphore`.

Pillar 4: Model-Agnostic Design

Hardcoding tensor layouts or model shapes limits maintainability across model generations.

- **GGUF / Safetensors Metadata Ingestion:** Parse standard model metadata at load time (e.g., dimension d, intermediate dimension, number of attention heads H_q, key/value heads H_{kv}, and rotary embedding base θ).
- **Declarative Compute Graph:** Instead of hardcoded forward passes, define the model as an acyclic graph of tensor operations (`RMSNorm`, `RoPE`, `GEMV`, `GEMM`, `SwiGLU`, `PagedAttention`).
- **Parametric Shader Library:** Keep dequantization logic independent of the operator graph. Compile shaders parameterized by quantization formats (`Q4_0`, `Q4_K`, `IQ3_S`, `Q8_0`, `FP16`) so weight formats can be swapped without touching the execution scheduler.

Phased Implementation Roadmap

```
Phase 1: Vulkan Tensor & Compute Harness
    ├── Device initialization & VMA memory pools
    ├── Quantized dequant/GEMV shaders (Q4/IQ3)
    └── Standalone GGUF model loader & linear forward pass
Phase 2: PagedAttention & Block Memory Manager
    ├── 16/32-token PagedAttention compute shader
    ├── Block allocator & Page Table indexing
    └── Multi-sequence decode validation
Phase 3: Hierarchical KV Tiering & Multi-GPU Pipelining
    ├── Dedicated transfer queue streaming (VRAM <-> Host RAM)
    ├── Heterogeneous pipeline split (MI50 + RX 6950 XT)
    └── Timeline semaphore synchronization
Phase 4: Agent Scheduling & Continuous Batching
    ├── Radix tree prompt prefix caching
    ├── Iteration-level continuous batcher
    └── Agent state machine (Tool-wait evictions)
```

Phase 1: Vulkan Core & Model-Agnostic Graph Runtime

1. **Memory Allocator:** Set up the Vulkan Memory Allocator (VMA) to manage `VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT` pools on both GPUs.
2. **SPIR-V Kernels:** Write baseline compute shaders for:
    - **Dequantization-GEMV:** Vector-matrix multiplication for decoding (optimized for memory bandwidth).
    - **Elementwise Ops:** `RMSNorm`, `SiluAndMul` (SwiGLU), and `RoPE` (Rotary Position Embeddings).
3. **Graph Execution:** Implement a clean loader that parses any standard GGUF file, builds a sequential operator list, and runs single-stream generation on one GPU.

Phase 2: PagedAttention & Quantized KV Engine

1. **Vulkan PagedAttention Kernel:** Write a compute shader that takes queries Q and reads non-contiguous Key/Value pages via an index buffer:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK_{\text{paged}}^T}{d_k}\right)V_{\text{paged}}$$

2. **Quantized KV Buffers:** Implement 4-bit (`Q4_0`) and 8-bit (`Q8_0`) packing directly inside the physical KV pages to cut context memory by 50% to 75%.
3. **Block Table Allocator:** Build a host-side free-list memory manager that distributes 16-token page IDs to active sequences on demand.

Phase 3: Hierarchical Tiering & Heterogeneous Pipeline

1. **Asynchronous Memory Engine:** Build a transfer worker utilizing `VK_QUEUE_TRANSFER_BIT`. Use timeline semaphores so the GPU can signal "Block X is no longer needed in this step," immediately letting the host queue a transfer to CPU RAM.
2. **Inter-GPU Pipeline Split:**
    - Place embeddings, first half of transformer layers, and their KV pool on the MI50 (`gfx906`).
    - Place the second half of transformer layers, final norm, LM head, and their KV pool on the RX 6950 XT (`gfx1030`).
    - Transfer intermediate activations across the PCIe bus via host-visible ring buffers.

Phase 4: Parallel Agent Engine & Continuous Batching

1. **Radix Tree Cache:** Store token sequences in a prefix tree. When an agent starts with an already-seen prompt, resolve its tokens directly to existing block IDs without running prefill.
2. **Continuous Batch Scheduler:** Process dynamic batches where finished sequences or tool-blocked agents drop out and new prompt tokens join every decode step.
3. **Cold Storage Integration:** Automatically migrate blocks to L2/L3 whenever an agent issues a tool call, bringing total concurrent agent capacity from 2 to 8+ on your 32GB total VRAM footprint.

Would you like to start by writing the Vulkan initialization and specialization-constant setup for your two cards, or would you prefer the layout and shader design for the PagedAttention kernel?

User prompt: I'd like to expand this in the following ways, designated what would be the best language to do this in (c++, rust or go) with the best framework for it's runtime and program package setup. Then for context expansion I'd like the qualities and options of other llama.cpp forks like beellama, turboquant (which is mostly in the mainline llama.cpp), rotor quant and llama.cpp mainline and how this ties into the kvcache cold storage system you defined above. Lastly for work concurrency, I need it expanding so there's more options for attention systems like the sliding windows of Gemma models, sparse attention from others and newer builds that could be multipurpose when calling and recalling context inside the actual kvcaching system in it's tiered approach, including the newer engram support as well as building extensibility for engrams and specific cold storage (read only) injection systems

Response: Language, Runtime, and Package Setup

For a project spanning low-level GPU hardware abstraction, asynchronous OS-level I/O, dynamic graph execution, and multi-agent network scheduling, **Rust** is the best choice.

- **Language Comparison:**
    - **C++20/23:** High peak performance and native compatibility with GPU APIs, but lacks safety guarantees. Building lock-free multi-tiered memory managers, asynchronous prefetch rings, and concurrent agent state machines in C++ frequently leads to segmentation faults, race conditions, and memory leaks.
    - **Go:** Strong for web scheduling and simple agent orchestration, but unsuitable for this runtime. Its garbage collector causes latency spikes during tensor ops, and CGo bindings introduce call overhead that degrades iterative token-level execution.
    - **Rust:** Zero-cost abstractions, memory safety without a garbage collector, and strong native support for asynchronous pipelines.
- **Core Framework Stack:**

- **GPU Driver Layer (`ash` / `vulkano`):** Use `ash` for explicit, 1:1 control over Vulkan API calls, descriptors, and timeline semaphores.
- **Memory Management (`gpu-allocator`):** Fast, production-grade sub-allocations for Vulkan device buffers (`VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT`) and host-coherent staging pools.
- **Async Engine & IO (`tokio` + `tokio-uring` / `io_uring`):** Tokio handles async agent loops, network endpoints, and queue arbitration. `io_uring` enables kernel-bypass asynchronous Direct I/O (`O_DIRECT`) reads and writes between the host NVMe drive and pinned host RAM.
- **Shader Pipeline (`slang-rs` or `naga`):** Write kernels in high-level HLSL/Slang or GLSL, compiling down to cross-architecture SPIR-V at build or run time.

```
[Agent Client / Server Session]
              │ (Tokio Async Task)
              ▼
    [Agent Workload Engine]
  (State machine, Radix-Tree Prefix, Sliding-Window router)
              │
              ▼
    [Paged KV Block Cache]
┌──────────────────────────────────────────────────┐
│ Layered Virtual Memory Page Table                │
├──────────────────────────────────────────────────┤
│ Tier 1: Device VRAM (MI50 / RX 6950 XT)          │ <--- ash (Vulkan Timeline Semaphores)
│ Tier 2: Host Pinned RAM (Staging Allocator)      │ <--- gpu-allocator
│ Tier 3: Local NVMe Storage (Zero-Copy mmap)      │ <--- tokio-uring (O_DIRECT)
└──────────────────────────────────────────────────┘
```

Context Compression & Upstream Fork Analysis

Standard FP16 or plain Q4/Q8 KV-cache representations consume substantial VRAM and scale poorly under long context windows. The modern `llama.cpp` ecosystem offers several alternatives for compressing this memory footprint:

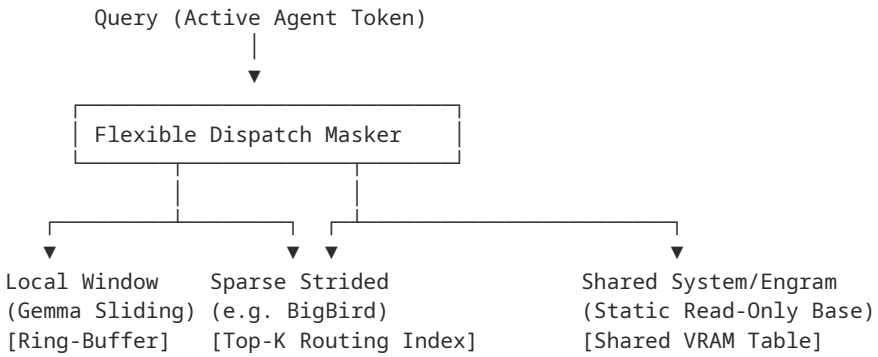
Implementation / Fork	Mathematical Mechanism	Strengths	Weaknesses	Tiered KV Integration Fit
Mainline `llama.cpp`	Scalar block-quant (`Q8_0`, `Q4_0`, `Q4_K`)	Baseline standard; robust cross-platform fallback; stable across hardware.	High quantization error on Outliers at 2–3 bit; significant PPL degradation below 4-bit.	Simple byte layouts; easy to serialize directly to NVMe or swap to Host RAM.
TurboQuant (Google ICLR / Upstream integration)	Randomized Walsh-Hadamard Transform (WHT) + Lloyd-Max scalar quant	Rotates key/value vectors to remove coordinate correlations; enables 3-bit / 4-bit cache with minimal PPL hit.	WHT has $O(d \log d)$ overhead, adding compute load to decode/prefill loops.	Good for Tier 1 VRAM; compact footprint lowers PCIe transfer latency to Tier 2.
RotorQuant (IsoQuant / PlanarQuant)	Clifford Rotors / 2D Givens & 4D Quaternion block rotations	$O(d)$ compute complexity; parallelizable; avoids butterfly networks; preserves directional vector properties.	Requires uniform head dimension divisibility (e.g., $d = 128$).	Ideal for Tier 1 VRAM. Faster to dequantize on older GCN/RDNA architectures than WHT.
BeeLlama (`beellama.cpp`)	**KVarN** (Hadamard rotation + variance normalization) + **Precision Tail**	Combines 2–4 bit compression for older context tokens with uncompressed/FP16 "tail" tokens for recent context.	Complex split-cache bookkeeping.	**Best architectural fit.** Compressed bodies map cleanly to Tier 2/3, while active tails live in Tier 1.

Architectural Integration with the 3-Tier Storage Engine

1. **The Invariant Tail Concept:** Integrate BeeLlama's precision tail into the paged storage engine. Store the most recent N tokens (e.g., 256 or 1024 tokens) in Tier 1 VRAM as native FP16/BF16.
2. **Asymmetric Decay on Eviction:** When a KV block ages out of the active window or moves from Tier 1 (VRAM) to Tier 2 (Pinned RAM):
   - Run a lightweight compute pass using **RotorQuant (IsoQuant)** to project the vectors into 2-bit or 3-bit space.
   - Decreasing the payload size from 16 bits to 2–3 bits drops host-to-device PCIe bandwidth pressure by **80–87%**.
3. **Decoupled Key vs. Value Handling:** Because Key vectors determine routing attention weights and are more sensitive to distortion than Value vectors, retain Keys at 4-bit (`Q4_0` or `Iso4`) while crushing Values down to 2-bit (`Iso2` or `Q2_0`).

Concurrency, Flexible Attention, and Cold Storage Injection

Modern multi-model support requires decoupling the attention engine from fixed full-causal attention matrices to handle sliding windows, sparse graphs, and read-only memory caches.



1. Unified Modular Attention Descriptors

Instead of hardcoding full multi-head attention loops in SPIR-V, compile a single **PagedAttention Dispatch Shader** conditioned on a runtime attention mask struct:

```
#[repr(C)]
struct AttentionDescriptor {
    head_dim: u32,
    sliding_window_size: u32, // 0 = Full Causal, >0 = Gemma-style sliding window
    sparse_stride: u32,        // 1 = Dense, >1 = Strided global attention
    block_size: u32,           // Typically 16 or 32 tokens
    is_read_only_injection: u32, // 1 = Bypass position shift, treat as global memory
}
```

- **Sliding Window (Gemma-style):** Constrain block-table traversal. The thread group computing attention for a token at position T skips loading block indices corresponding to positions $< (T - \text{sliding_window_size})$. This avoids loading older KV blocks from memory.
- **Sparse Attention:** Maintain a lightweight, 1-bit sparsity bitmask per sequence. The compute shader queries the bitmask; if a 16-token tile evaluates to zero, it bypasses the corresponding global VRAM fetch entirely.

2. DeepSeek-Style Engram Support & Static Cold Memory Injection

The **Engram architecture** introduces static, conditional N-gram memory tables accessed via hashing rather than transformer layer computation. This design pattern can also be used for **read-only context injection**:

- **How Engram Operates:**
  - Suffix tokens are normalized (NFKC/lowercased) and run through multi-head multiplicative-XOR hashes.
  - These hashes point directly to rows in a large, pre-compiled embedding/memory table.
  - The retrieved memory vectors are fused directly into the transformer activations via learned gating, bypassing multi-layer KV-cache growth for static patterns.
- **Cold Storage Injection (Injectable Read-Only Blocks):**
  - **Static Prefix Sharing:** Agent system prompts, documentation chunks, and API tool specifications are marked as **Read-Only Injection Blocks**.
  - **Zero-Copy Reference:** These blocks are initialized once, assigned a negative sequence ID (e.g., `Seq -1`), and permanently pinned in high-speed VRAM.
  - All running agents link their page tables to these blocks. The attention kernel resolves their positions via an offset translation vector:

$$\text{pos}_{\text{eff}} = \begin{cases} \text{pos}_{\text{table}} & \text{if block is read-only injection} \\ \text{pos}_{\text{sequence}} & \text{otherwise} \end{cases}$$

  - When an agent context is evicted to NVMe, these shared read-only blocks are never serialized to disk; only the agent-specific delta block tables are written.

Step-by-Step Implementation Strategy

```
Step 1: Core Engine Skeleton (Rust + Vulkan)
 ├── Initialize Vulkan physical devices (MI50 + RX 6950 XT)
 ├── Wrap memory allocation with `gpu-allocator`
 └── Implement timeline semaphore primitives for async execution
Step 2: PagedAttention + RotorQuant SPIR-V Kernel
 ├── Develop parametric compute shader (Wave32 / Wave64 specialization)
 ├── Integrate IsoQuant 4D Givens rotation dequantizer
 └── Validate Gemma-style sliding window mask in SPIR-V
Step 3: Multi-Tier KV-Engine & IO-Uring Backend
 ├── Build host-side Block Table Manager (Allocation, Eviction, Ref-Counting)
```

```
 ├── Integrate `tokio-uring` for asynchronous Direct I/O swapping to NVMe
 └── Implement background transfer queue for VRAM <-> Pinned RAM staging
Step 4: Engram Table & Multi-Agent Scheduler
 ├── Build tokenizer compression & multiplicative-XOR hash engine
 ├── Add read-only injection table handling into the attention dispatch pass
 └── Write the continuous batching scheduler for parallel agent state loops
```

Step 1: Rust System Bootstrap with Vulkan

Set up the dual-device Vulkan initialization in Rust. Query `VkPhysicalDeviceProperties` to detect whether each device needs Wavefront 32 (RX 6950 XT) or Wavefront 64 (MI50), and inject this into SPIR-V specialization constant `0`.

Step 2: The Parametric Attention Shader

Write the compute kernel using Slang or raw GLSL. Structure it around block lookups rather than a flat buffer. Implement the **RotorQuant** decode logic directly in the dequantization prologue of the shader to support 2-bit and 4-bit packed tokens.

Step 3: Tiered Eviction Worker

Implement a background Tokio task managing the KV block pool:

- Monitor GPU memory pressure. If device allocation passes 90%, flag the oldest blocks using an LRU policy (excluding read-only injection blocks).
- Submit an asynchronous copy command on the `VK_QUEUE_TRANSFER_BIT` to move candidate blocks to pinned host RAM.
- If an agent enters `WaitingForTool`, use `io_uring` to flush those blocks from pinned RAM directly to an NVMe backing file, releasing host memory.

Step 4: Engram Integration

Implement the hash lookup routine in Rust. At startup, load the static Engram table directly into host-pinned RAM or split across remaining VRAM. Connect the hash lookups to the forward pass prior to layer processing, passing the combined hidden state into your heterogeneous pipeline.

User prompt: Excellent details here - can you think of any other ideas we could add to this? Any sort of streamlined management system, agnostic model format systems and any other general performance increases you can think of? Maybe something with tailnet systems in mind that spreads models over a standard network that we could take advantage of with our tiered compression systems along with model weights swapping systems?

Response: 1. Tailnet-Aware Distributed Mesh & Networked KV Tiering

Integrating your tiered memory system with a tailnet (WireGuard/Tailscale mesh) allows multiple heterogeneous workstations, edge nodes, or cloud instances to pool VRAM and storage without dedicated cluster interconnects like InfiniBand.

```
[Agent / Client Node]
         │ (Tailnet WireGuard Tunnel)
         ▼
[Ingress & Orchestration Node]
  ├── Local Hot Layers (RX 6950 XT Compute)
  └── Dispatches compressed intermediate activations over Tailnet (QUIC / UDP)
         │
         ▼
[Worker Node (e.g., Remote MI50 / Headless Rig)]
  ├── High-Bandwidth Layers & MoE Experts (HBM2)
  └── Tier 4: Remote KV Eviction Pool / Peer-to-Peer NVMe Share
```

Tier 4: Networked Cold KV Eviction

- **UDP/QUIC-Backed Block Streaming (`quinn` or `s2n-quic`):** Treat the network as **Tier 4** in your storage hierarchy (below local NVMe). When an idle agent's context is evicted from a local device, stream compressed blocks over QUIC directly to a storage-heavy headless node.
- **Over-the-Wire RotorQuant Compression:** Never transmit uncompressed FP16 activations or KV states across a tailnet. Applying RotorQuant / IsoQuant (reducing vectors to 2-bit or 3-bit representations) reduces the network payload of an attention state from 8 KB to ~1 KB per token vector. This makes continuous pipeline parallelism and cross-node KV migration feasible over residential internet or mesh connections.
- **Agent Context Migration (Follow-the-Work):** If an agent requires extensive external tool execution that runs on a remote server, serialize only the active block-table pointers and stream the delta KV blocks over the tailnet. The remote node resumes the agent's decode loop locally without re-running prefill.

Networked Weight Swapping & Content-Addressable Storage (CAS)

- **Chunk-Level Hash Addressing (BLAKE3):** Partition model weights and MoE expert blocks into deterministic chunks (e.g., 4 MB to 16 MB blocks) indexed by BLAKE3 hashes.
- **Peer-to-Peer Delta Syncing:** When upgrading a model checkpoint (e.g., fine-tunes or base-model revisions), nodes on the tailnet query peer chunk registries. Identical layers (like standard attention blocks shared between base and instruct variants) are resolved locally, streaming only changed weight blocks across the network.

2. Agnostic Model Format & Virtual Tensor Container (VTC)

Avoid baking format-specific parsers directly into execution graphs. Implement a format-agnostic abstraction layer:

```
[Physical Source Files]
(GGUF, SafeTensors, Raw Binaries, Remote HTTP/CAS)
           │
           ▼
[Virtual Tensor Container (VTC)]
  ├── Zero-Copy Mmap Memory Map
  ├── Format-Agnostic Tensor Descriptor (Shape, DType, Quant Layout)
  └── Dynamic Transpilation Engine (On-the-fly Tiling & Permutation)
           │
           ▼
[Vulkan Runtime Engine]
  (Dispatches straight to device buffers regardless of source format)
```

- **Decoupled Tensor Descriptors:** A lightweight metadata header maps logical layer paths (`model.layers.0.mlp.down_proj.weight`) to physical descriptors containing byte offsets, raw quantization formats, stride layouts, and memory sources (local file slice, remote URI, or memory-mapped buffer).
- **Zero-Copy Hybrid Loading:**
  - Map SafeTensors, GGUF, or raw binary checkpoints simultaneously into the same virtual address space using `memmap2`.
  - If a model has FP8/FP16 weights in SafeTensors and a quantized MoE router in GGUF, the engine loads both into the unified DAG without requiring an offline format conversion step.
- **Dynamic On-The-Fly Swizzling Shaders:** Hardware targets prefer specific tensor layouts (e.g., tile-swizzling for RDNA 2 Wave32 vs. linear formats for GCN Wave64). Write lightweight one-shot Vulkan compute shaders that run during weight ingestion, restructuring raw source tensors into hardware-optimal cache layouts as they are copied into device VRAM.

3. Dynamic Runtime Management & S-LoRA Swapping

Managing multiple agents concurrently requires an orchestration layer capable of dynamic prioritization and hot-swapping task adapters.

- **Dynamic VRAM Governor & Proactive Eviction:**
  - Track a three-tier memory watermark: `Nominal`, `Pressure`, and `Critical`.
  - When device VRAM enters `Pressure` (e.g., >85% capacity), the runtime automatically triggers non-blocking DMA copies of the least recently used (LRU) inactive agent blocks to Host RAM (Tier 2).
  - If a sudden memory spike hits `Critical` (>95%), decode tasks for low-priority agents pause mid-step while their blocks flush to Tier 3 (NVMe) via `io_uring`, preventing Vulkan allocation failures (`VK_ERROR_OUT_OF_DEVICE_MEMORY`).
- **S-LoRA Style Multi-Adapter Swapping:**
  - Different agents often require specialized behaviors (e.g., SQL generation, creative drafting, strict JSON emission).
  - Keep the base model weights frozen in VRAM. Maintain small low-rank adapter matrices (A and B, typically <50 MB each) in a dedicated host-pinned ring buffer.
  - Dynamically bind adapter descriptors per agent slot. The attention and MLP compute shaders evaluate the shared base projection and add the agent-specific adapter contribution in-place:

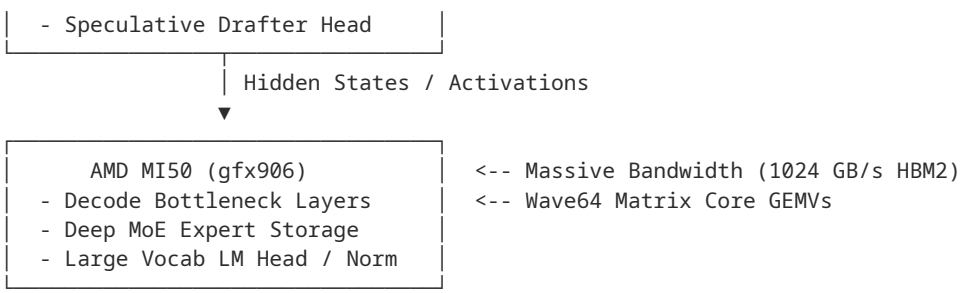
$$Y = X \cdot W_{\text{base}} + (X \cdot A_{\text{agent}}) \cdot B_{\text{agent}} \cdot \alpha$$

- **Observability Control Plane:** Embed an asynchronous Prometheus/OpenTelemetry exporter and a lightweight local management API. Expose metrics like cache-hit ratios across Tiers 1–4, active agent slots, PCIe DMA transfer queue saturation, and per-device compute utilization.

4. Heterogeneous Hardware & Architectural Optimizations

To extract maximum throughput from mismatched AMD silicon, tailor execution patterns to the specific physical strengths of each GPU.

```
      Token Input / Prefill Phase
                 │
                 ▼
┌──────────────────────────────────┐
│   AMD RX 6950 XT (gfx1030)       │  <-- High Compute Clocks (2.3+ GHz)
│  - Prefill Matrix Engines        │  <-- 128 MB Infinity Cache (Fast scratchpad)
```



Asymmetric Asynchronous Pipeline Splitting

- **Assign Memory-Bound Tasks to the MI50:** The MI50 features 1024 GB/s of HBM2 bandwidth—nearly double the 576 GB/s of the RX 6950 XT. Route memory-intensive operations (batch-1 decode iterations, large vocabulary projection layers, and primary MoE expert layers) to the MI50.
- **Assign Compute-Bound Tasks to the RX 6950 XT:** The RX 6950 XT features substantially higher boost clocks and 128 MB of high-speed Infinity Cache. Use it for the prefill attention computation, speculative draft token proposals, and dynamic quantization/dequantization operations.
- **Zero-Copy Host Pointers (`VK_EXT_external_memory_host`):** Enable `VK_EXT_external_memory_host` to bind host-pinned RAM directly into the GPU address space. This allows the GPU to stream or read host-resident expert weights without allocating an intermediate staging buffer, cutting memory overhead in half.

SPIR-V Kernel Fusions

Chained operations in standard LLM inference waste bandwidth by constantly writing intermediate results back to VRAM. Combine them into single-pass kernels:

- **Fused RMSNorm + QKV Projection:** Read the hidden state once, calculate the root-mean-square normalization within registers, apply scaling, and immediately run the matrix-vector multiplication for Query, Key, and Value projections in local LDS (Local Data Share).
- **Fused SwiGLU / GeGLU:** Compute Gate and Up projections concurrently in register tiles, evaluate the activation function ($\text{silu}(x) \cdot y$), and stream the result directly into the Down projection accumulator, eliminating three intermediate memory round-trips.

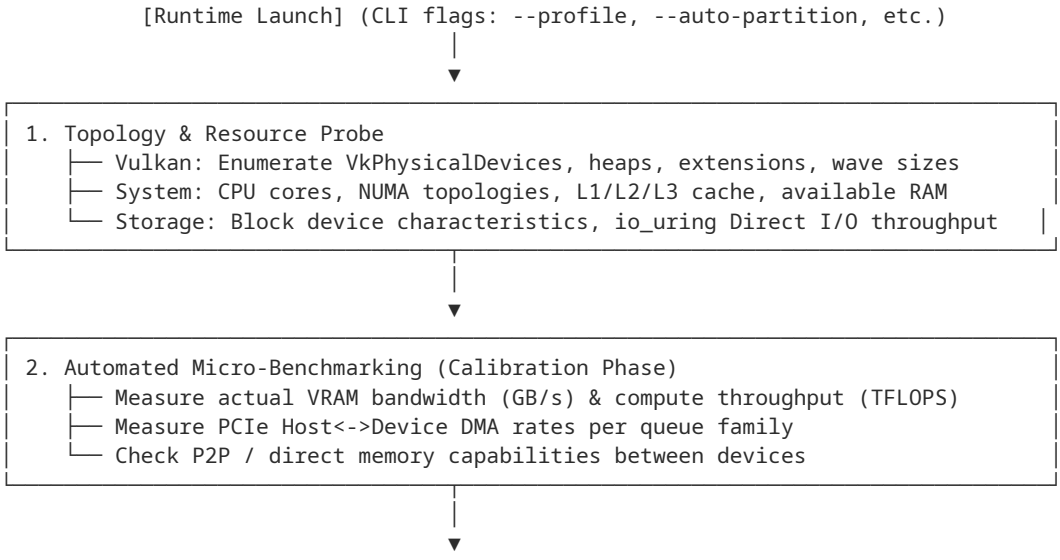
Speculative Multi-Agent Tree Verification

Instead of running speculative decoding for a single stream, run a **Tree-Speculative Dispatcher**. A small, fast drafter model running on the RX 6950 XT proposes a tree of candidate tokens across active agents. The MI50 evaluates the valid branches across all agents in a single batched compute pass, turning memory-bound serial decode steps into compute-bound parallel matrix evaluations.

User prompt: Excellent - can there be a setup/initialisation system baked into this that can be triggered/retriggered to have an identifying system on where and what were making use of? So your example for the bandwidth bound work on the mi50, and the compute bound on the rx, could we sort of have some launch flags that tells the runtime that it has access to storage, ram, couple cores, cache, gpu's and any other forms of compute, it goes off and identifies and maps it out from what's been identified to make use of and then dynamically assigns the model weights? From your suggestion it would be building it up, but it would only be building it for my system and would need updating when there are hardware changes

Response: An autonomous initialization and calibration system solves the portability problem. Instead of hardcoding layout rules for your specific hardware, the runtime boots into a **Hardware Discovery & Topology Solver** phase that inspects, micro-benchmarks, and dynamically partitions model weights and KV buffers across whatever devices it finds.

System Architecture: The Discovery-to-Execution Pipeline



```
┌──────────────────────────────────────────────────────────────────────┐
│ 3. Cost Model & Graph Solver (Dynamic Partitioning)                  │
│    ├── Classify hardware (Compute-heavy vs. Bandwidth-heavy nodes)   │
│    ├── Allocate model layers, drafter heads, and MoE experts         │
│    └── Size Tier 1 (VRAM), Tier 2 (Pinned RAM), and Tier 3 (NVMe) KV pools │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  │
                                  ▼
                  [Hardware Plan Emitted / Engine Executes]
```

Step 1: Hardware Discovery & Topology Probing

On startup, the engine queries the operating system and graphics drivers directly through low-level APIs.

- **Vulkan Device Discovery (`ash`):**
  - Enumerates each `VkPhysicalDevice`.
  - Inspects `VkPhysicalDeviceMemoryProperties` to record discrete VRAM capacities (`VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT`) and host-coherent staging apertures.
  - Queries `VkPhysicalDeviceSubgroupProperties` to detect whether each device runs natively on Wave32 (RDNA/NVIDIA) or Wave64 (GCN/CDNA).
  - Checks extensions: `VK_KHR_timeline_semaphore` for asynchronous synchronization, `VK_EXT_external_memory_host` for zero-copy CPU pinned memory mapping, and `VK_KHR_cooperative_matrix` for hardware tensor units.
- **CPU & Cache Topology (`hwloc2` / `sysinfo` in Rust):**
  - Detects core count, physical threads, and NUMA node layout.
  - Queries L1, L2, and L3 cache boundaries. If the system has large L3 caches (e.g., AMD 3D V-Cache), those CPU cores can be assigned to latency-sensitive token scheduling and drafter loops.
- **NVMe & Storage Inspection:**
  - Checks designated cache mount points for Tier 3 cold storage using Linux `sysfs` or Windows device IO.
  - Verifies support for direct kernel-bypass I/O (`O_DIRECT` via `io_uring` on Linux or Overlapped I/O on Windows).

```rust
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct DeviceProfile {
    pub device_id: u32,
    pub name: String,
    pub pci_bus_id: String,
    pub total_vram_bytes: u64,
    pub usable_vram_bytes: u64,
    pub native_wavefront: u32,
    pub measured_bandwidth_gbps: f32,
    pub measured_compute_tflops: f32,
    pub pcie_h2d_bandwidth_gbps: f32,
    pub pcie_d2h_bandwidth_gbps: f32,
    pub role: DeviceRole, // BandwidthBound, ComputeBound, or Unified
}
```

Step 2: Active Micro-Benchmarking (The Calibration Run)

Paper specifications often misrepresent true throughput due to PCIe lane bifurcation, power limits, thermal throttling, or driver quirks. When the engine calibrates, it runs isolated, lightweight SPIR-V kernels for 2–3 seconds across all discovered resources:

1. **Memory Bandwidth Sweep:** Runs a strided vector-read/write compute shader to calculate real-world memory transfer speeds. (Your MI50 will post closer to ~800–900 GB/s actual HBM2 bandwidth, while the RX 6950 XT will hit ~450–500 GB/s GDDR6).
2. **Compute Arithmetic Intensity Test:** Dispatches a fused multiply-accumulate (FMA/GEMM) workload. (The RX 6950 XT will outscore the MI50 here thanks to higher core clocks and RDNA dual-issue compute units).
3. **PCIe DMA Transfer Probe:** Submits asynchronous memory copy commands over `VK_QUEUE_TRANSFER_BIT` using pinned host memory to determine actual Host ↔ Device throughput per card.
4. **Direct I/O Storage Calibration:** Writes and reads temporary 64 KB aligned blocks through `io_uring` to verify sustained disk read/write bandwidth for Tier 3 KV paging.

Step 3: The Cost Model & Graph Solver

Once metrics are recorded, a solver dynamically assigns tasks based on arithmetic intensity:

$$\text{Operational Intensity} = \frac{\text{FLOPs Required}}{\text{Bytes Transferred}}$$

- **Prefill & Speculative Drafters $\rightarrow$ Assigned to High-Compute Devices:**
  - Prompt processing (prefill) and speculative draft model evaluation have high arithmetic intensity.
  - The solver routes prefill passes and draft token proposals to the device with the highest measured TFLOPS (e.g., the RX 6950 XT).
- **MoE Experts, Decode Layers, & Large LM Heads $\rightarrow$ Assigned to High-Bandwidth Devices:**
  - Single-token decode and dynamic MoE expert streaming are bandwidth-bound ($O(1)$ operations per byte transferred).
  - The solver assigns dynamic expert cache slots and deep sequential decoder layers to the device with the highest measured GB/s (e.g., the MI50).
- **Automated Tier Allocation:**
  - **Tier 1 (Device VRAM):** The solver reserves space for model weights, working activations, and an initial pool of high-priority KV pages.
  - **Tier 2 (Pinned Host RAM):** Sized based on total unallocated system RAM (e.g., reserving 4–8 GB for the OS, claiming 70% of the remainder for the staging pool).
  - **Tier 3 (Local NVMe):** Configured based on available disk space at the specified directory.

Step 4: CLI Flags, Persistent Profiles, and Invalidation

To make the initialization friction-free, use a persistent hardware fingerprinting pattern:

CLI Control Flags

```
# Force a full recalibration of all compute, memory, and disk resources
./runtime --reprofile

# Run with dynamic autotuning based on existing profile
./runtime --model qwen3.8-fn.gguf --auto-partition

# Print the dynamic layout plan and exit without running inference
./runtime --model qwen3.8-fn.gguf --dry-run-plan

# Explicit manual overrides if the user wants to force specific behavior
./runtime --override-role gpu:0=bandwidth --override-role gpu:1=compute --tier2-ram 32GB
```

Hardware Fingerprinting & Invalidation

1. **Fingerprint Generation:** Compute a hash of all hardware signatures on startup:

$$\text{System Hash} = \text{BLAKE3}(\text{GPU UUIDs} + \text{Driver Versions} + \text{PCIe Bus IDs} + \text{Total RAM} + \text{CPU Model})$$

2. **Profile Persistence:** Save the hardware profile and benchmark results to `~/.config/inference_engine/hw_profile.json`.
3. **Automatic Re-triggering:**
   - On normal boot, the runtime hashes the current system.
   - If the hash matches `hw_profile.json`, it skips the micro-benchmarks and starts immediately.
   - If the hash differs (e.g., you swapped a GPU, added RAM, or updated the graphics driver), the engine logs: `"Hardware change detected. Running automated calibration pass..."` It re-runs the micro-benchmarks, updates the JSON profile, recalculates the layer distribution, and continues execution.

Example Solver Output (`--dry-run-plan`)

```
[INFO] Hardware Fingerprint: a8f4b72e19... (Cached)
[INFO] Discovered Devices:
  - Device 0: AMD Radeon Instinct MI50 (16 GB HBM2 | 874 GB/s measured | 13.2 TFLOPS) -> Assigned: Bandwid
  - Device 1: AMD Radeon RX 6950 XT (16 GB GDDR6 | 482 GB/s measured | 21.4 TFLOPS) -> Assigned: Compute W
  - Host RAM: 64 GB Total (Allocating 36 GB Pinned Pool for Tier 2 KV Cache)
  - Storage: Samsung 980 Pro NVMe (Allocating 250 GB for Tier 3 Cold Swapping via io_uring)

[INFO] Dynamic Model Partitioning for Qwen 3.8 Flash Next:
  - Speculative Drafter Head : Device 1 (RX 6950 XT) [Low latency compute]
  - Transformer Layers 0-14  : Device 1 (RX 6950 XT) [Prefill optimized]
  - Transformer Layers 15-32 : Device 0 (MI50)       [HBM2 bandwidth decode]
  - Active MoE Expert Cache  : Device 0 (MI50)       [High-speed expert paging]
```

```
  - Tier 1 KV Capacity       : 1,800 active blocks split across VRAM
  - Tier 2 KV Capacity       : 18,400 warm blocks in Host RAM
  - Tier 3 KV Capacity       : Uncapped cold storage on NVMe (Prefetch-enabled)
```

User prompt: And how would that be expanded to be tailnet capable - there would have to be a setup of node devices across relevant hardware right? This identifier needs to be able to work agnostically across the connected nodes so it's able to work on any device - and the system needs to be able to have a failure system in place should something drop off the network. It should also cope with systems that change their connectivity (heaven forbid wireless or preferably wired, say a laptop oflr phone gets moved it shouldn't just take it out of the setup and corrupt the entire inference system)

Response: To expand this runtime across a tailnet (Tailscale, Headscale, or pure WireGuard mesh) without risking inference corruption or pipeline freezes when nodes drop or change connections, the architecture must transition from a **static compute pipeline** to an **asynchronous, lease-based distributed actor system**.

Here is the design for node discovery, transport resilience, and fault-tolerant scheduling across dynamic hardware.

1. Unified Node Daemon & The Distributed Manifest

Every participating machine runs a lightweight binary (`nodectl` / `infer-daemon`). Whether running on an unthrottled dual-GPU desktop, an M-series MacBook, a headless server, or a mobile device, the daemon executes the same local probe and broadcasts a standardized **Node Manifest**.

```
[Anchor Node (Your Rig: MI50 + RX 6950 XT)]
  └── Runs Coordinator & Local Graph Scheduler
            │
            ├── (Tailnet Mesh via QUIC)
            │
  ┌─────────┴──────────────────────────┬──────────────────────────┐
  ▼                                    ▼                          ▼
[Node B: Headless Server]     [Node C: Laptop]          [Node D: Phone / SBC]
- High RAM / Fast NVMe        - Transient Wi-Fi         - Ultra-low compute
- Tier 4 KV Storage Node      - Speculative Drafter     - Remote Token Sink
```

The Node Manifest Structure

Nodes register with the mesh by advertising capabilities, device constraints, and network stability tiers:

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct NodeManifest {
    pub node_id: Uuid,
    pub tailnet_ip: IpAddr,
    pub node_class: NodeReliabilityClass, // Anchor, StableWorker, or Transient
    pub compute_devices: Vec<DeviceProfile>, // Vulkan/Metal/CPU specs
    pub memory_tiers: NodeMemoryCapacity,  // Available VRAM, Host RAM, NVMe
    pub network_profile: NetworkMetrics,   // RTT, Jitter, Bandwidth, ConnectionType
}

#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum NodeReliabilityClass {
    Anchor,       // Fixed desktop/server on wired Ethernet (Hosts core graph & KV master)
    StableWorker, // Dedicated compute node with high uptime
    Transient,    // Laptops, Wi-Fi nodes, mobile devices (Restricted workloads only)
}
```

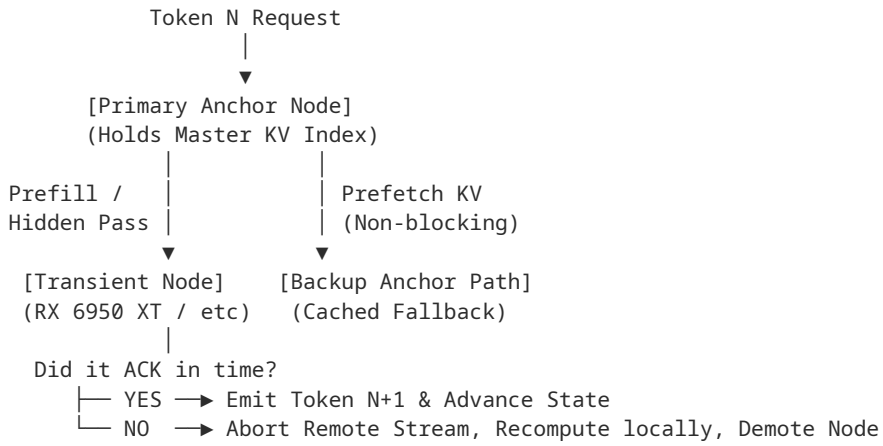
2. Transport Layer: Handling Moving Connections Without State Loss

Standard TCP connections break and stall application state when a laptop switches from Ethernet to Wi-Fi or roams across access points.

- **QUIC over WireGuard (`quinn`):** Run the inter-node transport entirely over QUIC (UDP) rather than TCP or gRPC.
    - **Native Connection Migration:** QUIC uses 64-bit Connection IDs rather than IP/port tuples. If a device drops Ethernet and switches to Wi-Fi, the IP changes, but the QUIC connection migrates seamlessly in flight without dropping active tensor transfers.
    - **Head-of-Line Blocking Elimination:** If a single intermediate layer packet drops over a jittery connection, only that specific tensor stream waits for retransmission; all other agent streams and KV paging blocks continue uninterrupted.
- **Continuous Adaptive RTT Probing:**
    - Background pingers continuously measure round-trip latency and packet loss.
    - If a node's latency spikes (e.g., someone closes a laptop lid or Wi-Fi signal degrades), its cost score in the scheduler drops instantly, triggering preemptive job re-routing before packets start timing out.

3. Fault Tolerance: Preventing Network Drops from Corrupting Inference

If an intermediate node in a naive pipeline parallel setup disappears, the entire forward pass stalls and the running agent context is destroyed. To prevent this, implement **State Asymmetry** and **Idempotent Token Stepping**:



A. Master KV Ledger Stays on Anchor Nodes

- Never store the *only* copy of an active agent's block table or KV state on a `Transient` node.
- **Lease-Based Tier 4 Storage:** When cold KV blocks are offloaded over the network to a remote node's RAM/NVMe, they are treated as an **ephemeral cache lease**, not authoritative storage.
- The local Anchor Node keeps a compressed copy (e.g., 2-bit RotorQuant) in local Tier 2 (Host RAM) or Tier 3 (NVMe) until the remote node confirms it has successfully written and pinned the block. If the remote node disconnects, the Anchor Node simply drops the lease and reads from its local storage.

B. Asynchronous Speculative Drafting (Failure-Immune)

- Offload non-critical tasks to transient devices. A Wi-Fi-connected laptop or secondary device is ideal for running **Speculative Draft Models**.
- **Failure Mode:** If the laptop drops off the network mid-generation:
  1. The Anchor Node's verification loop hits a short deadline (e.g., 25 ms).
  2. The anchor drops the draft request without stalling.
  3. The anchor falls back to standard single-token autoregressive decoding on its local GPUs.
  4. Inference continues without output corruption or context loss.

C. MoE Fallback Routing

- If MoE experts are sharded across the tailnet to conserve local VRAM:
  - When a remote expert node fails to return hidden states within the latency threshold, the Anchor Node skips that remote expert and routes the token activation through the **Shared/Dense Base Expert** resident in local VRAM, or dynamically switches to an alternate local quantization.
  - The forward pass completes with a minor degradation in perplexity for that single token, rather than terminating the process with an unhandled exception.

4. Dynamic Topology Solver & Live Node Hand-offs

The scheduler treats the cluster as a dynamic graph that adapts as nodes appear, degrade, or vanish.

```
pub enum NodeEvent {
    Joined(NodeManifest),
    Degraded { node_id: Uuid, new_rtt: Duration, packet_loss: f32 },
    GracefulLeave(NodeManifest),
    HardDrop(Uuid),
}
```

Graceful Departure Hand-off

When a laptop shuts down, disconnects, or the user enters `nodectl drain`:

1. The departing node stops accepting new layer dispatches or KV storage writes.
2. It pushes any modified or dirty KV cache blocks back to the Anchor Node using its remaining transfer bandwidth.
3. The coordinator updates the active execution plan to bypass the node and releases its allocations without interrupting ongoing agent steps.

Sudden Drop (Dead-Man Timeout)

If a device loses power or drops off the network without warning:

1. **Heartbeat Lease Expiry:** Nodes exchange 500 ms heartbeats via UDP. A missed heartbeat marks the node as `Suspicious`; 1,500 ms without response triggers `HardDrop`.
2. **Immediate Graph Recompilation:** The Anchor Node invalidates execution steps targeting that node's device ID.
3. **Execution Fallback:** If the node was processing an intermediate pipeline layer, the Anchor Node rolls back to the intermediate hidden state checkpoint generated prior to the dispatch and finishes the layer on local hardware (e.g., your MI50).

5. Implementation Roadmap for the Mesh Layer

```
Step 1: Network Mesh & Manifest Protocol
 ├── Implement `nodectl` daemon using `tokio` and `ash`
 ├── Embed node auto-discovery and heartbeat gossip via `quinn` (QUIC)
 └── Serialize/exchange `NodeManifest` benchmarks over the wire
Step 2: Networked KV Cache Tiering (Tier 4)
 ├── Implement remote block read/write commands over QUIC streams
 ├── Add RotorQuant compression/decompression pipeline for wire transfers
 └── Build lease manager with local fallback fallback-retention
Step 3: Dynamic Graph Partitioning & Failure Handlers
 ├── Adapt solver to weigh network RTT into operational intensity calculations
 ├── Implement speculative drafting client that degrades to local autoregression
 └── Add dead-man circuit breaker to rerun dropped pipeline steps locally
```

Step 1: Daemon & Manifest Exchange

Write a small Rust daemon that runs the local hardware probe built in the previous phase. When the daemon launches with `--connect <tailnet-coordinator>`, it opens a bidirectional QUIC stream, sends its `NodeManifest`, and begins sending periodic heartbeat pings.

Step 2: Remote Tier 4 KV Paging

Integrate network paging into the local `PagedAttention` block allocator:

- When local VRAM (Tier 1) and local RAM (Tier 2) are saturated, assign cold agent blocks to remote nodes that have advertised spare RAM.
- Compress blocks using 2-bit RotorQuant before sending them across the wire.
- Maintain a local bloom filter or hash table of block locations. If a query hits a remote block, request it asynchronously via QUIC while the agent's previous layers are evaluating.

Step 3: Resilient Graph Dispatcher

In the coordinator's execution loop, wrap remote network dispatches in a timeout future:

- If the remote execution completes, accept the tensor and proceed.
- If the remote execution times out or returns a connection error, trigger the fallback path: mark the node as offline, re-run the layer locally using the local GPU pipeline, and notify the continuous batcher to rebalance the next generation step.

User prompt: ok can you give me a few names for this that is mostly unique but isnt over the top?

Response: Layered & Geological (Nodding to memory tiers and ecosystem heritage)

- **Lamina** (`lamina`)
  - **Why it fits:** Geological and anatomical term for thin, stacked layers or sheets. It mirrors your 4-tier memory hierarchy (VRAM → Host RAM → NVMe → Tailnet), block-paged KV tables, and pipeline-split model layers across the MI50 and RX 6950 XT.
  - **CLI feel:** `lamina serve --auto-partition`
- **Talus** (`talus`)
  - **Why it fits:** The natural slope of rock debris forming a solid foundation at the base of a cliff. It subtly nods to the lineage of engines like *Strata* and *Basalt* without copying them, sounding like a grounded low-level systems utility.
  - **CLI feel:** `talus run --model qwen.gguf`
- **Breccia** (`breccia`, *breh-chee-uh*)
  - **Why it fits:** A stone composed of disparate, angular mineral fragments fused into a single resilient rock. It is an exact physical metaphor for binding heterogeneous, mismatched hardware (GCN Wave64 + RDNA Wave32 + remote mesh devices) into a unified engine.

- **CLI feel:** `breccia daemon --mesh-join`

Structural & Network (Nodding to parallel agents, paged blocks, and tailnets)

- **Tessera** (`tessera`)
  - **Why it fits:** The Latin name for the individual square tiles used to assemble a mosaic. A natural fit for a runtime built on PagedAttention blocks, discrete Engram tables, and distributed node nodes that together form one inference canvas.
  - **CLI feel:** `tessera init --profile`
- **Plexus** (`plexus`)
  - **Why it fits:** A braided network of vessels or nerves that share pathways and adapt dynamically. It reflects the multi-agent state machines, QUIC mesh routing, and Radix-tree prefix branching.
  - **CLI feel:** `plexus route --tailnet`
- **Stria** (`stria`)
  - **Why it fits:** Latin for parallel grooves or striations. Captures both the compute side (parallel SIMD wavefront execution across differing GPU lanes) and the runtime side (continuous batching of parallel agent streams).
  - **CLI feel:** `stria bench --devices all`

Minimalist & Systems-Level (Punchy, Unix-style tools)

- **Vane** (`vane`)
  - **Why it fits:** A single syllable, evocative of directional flow, turbine blades, and precision balance. It carries a subtle nod to **V**ulkan while capturing the streaming nature of your asynchronous PCIe and network queues.
  - **CLI feel:** `vane up`
- **Calx** (`calx`)
  - **Why it fits:** Short, sharp, and technical. Originating as the elemental core left after refinement, it has the feel of foundational systems software (like *nix*, *ash*, or *mmap*) suited for an engine stripped of CUDA bloat down to bare-metal SPIR-V.
  - **CLI feel:** `calx exec`

User prompt: breccia it is! can you make a sweep of assessments to make a plan for this project? i'd like it to work on the tailnet works LATER - right now i need it to focus on the vulkan backend and optimizations that just arent being done - im using qwen 3.8 flash next iq3 right now at a rather limiting 20tps, i want this better and for it to adapt to my system in the agnostic ways, along iwth the advanced integrations for cache streaming, model loading on the local machine and the ways that models can be supported along with context taht newer inference engines like infernix , strata, ninfer and others but focused around vulkan.

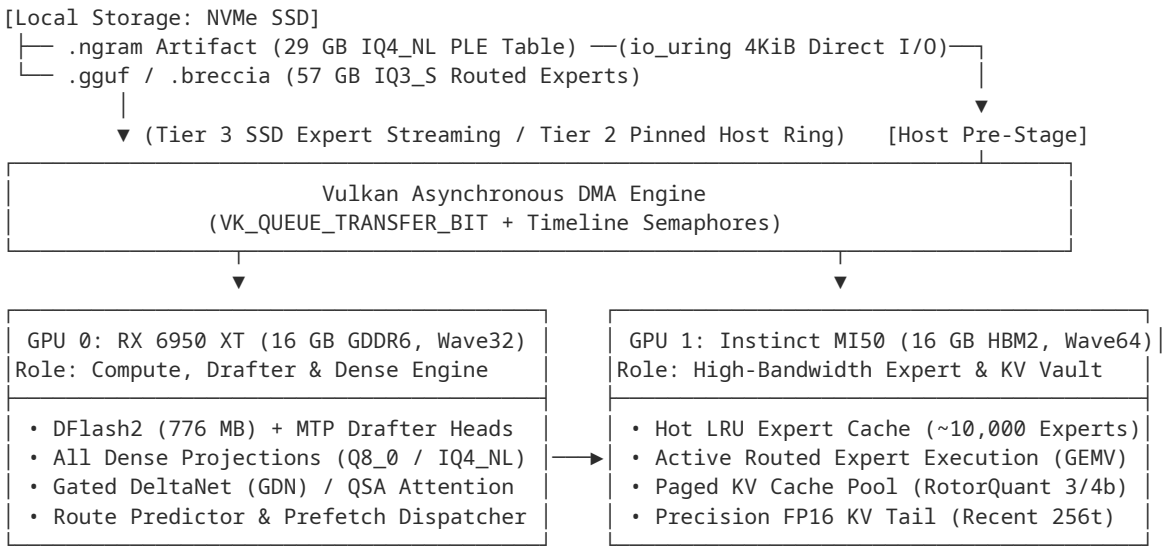
Response: 1. Technical Assessment: Why Your Setup Is Stuck at 20 TPS

Understanding the architectural gap between `llama.cpp` and specialized engines like **Infernix**, **Strata**, **Basalt**, and **ninfer** explains why `Qwen3.8-Flash-Next` (`IQ3_S`) bottlenecks at 20 TPS on your system:

- **Static Layer Splitting vs. Dynamic Expert Caching:** `Qwen3.8-Flash-Next` is a massive Mixture-of-Experts (MoE) + Per-Layer Embedding (PLE) architecture (~57 GB for the `IQ3_S` weights plus a ~29 GB `IQ4_NL` PLE n-gram table). `llama.cpp` statically offloads a fixed subset of full layers into your 32 GB total VRAM (16 GB MI50 + 16 GB RX 6950 XT) and forces the host CPU to compute the remaining layers sequentially. In contrast, **Infernix** and **Basalt** keep *100% of the dense attention weights and routers permanently in VRAM* on every layer, filling the remaining VRAM with a dynamic **LRU Expert Cache** (holding ~13,000 hot `IQ3_S` experts in VRAM for a 98.6% cache-hit rate) and streaming only cache misses over PCIe or from an NVMe SSD tier.
- **Synchronous PLE N-Gram Lookups:** `Qwen3.8-Flash-Next` relies on a massive PLE n-gram table. In `llama.cpp`, fetching these embeddings stalls the pipeline. Engines like **Infernix** separate the PLE volume into an external `.ngram` artifact read in random 4 KiB blocks via asynchronous direct NVMe I/O (`io_uring`), completely bypassing host RAM bottlenecks.
- **Missing Speculative Drafters (DFlash2 & MTP):** `llama.cpp` evaluates one token per forward pass. **Infernix** pairs the model's native Multi-Token Prediction (MTP) head with external **DFlash2** (a 776 MB drafter) and n-gram drafting, verifying 4 to 7 tokens in a single batched pass. Evaluating 5 tokens at once in an MoE model takes roughly the same memory bandwidth as evaluating 1 token, multiplying effective throughput by 3x–4x.
- **Unfavorable Vulkan Dispatch in `ggml-vulkan`:** `llama.cpp`'s Vulkan backend treats your MI50 (`gfx906`, Wave64, 1024 GB/s HBM2) and RX 6950 XT (`gfx1030`, Wave32, 576 GB/s GDDR6) as symmetric sequential layer workers, incurring massive host-synchronization overhead between layers and failing to use dedicated Vulkan DMA transfer queues for overlapping weight streaming.

2. Core Local Architecture of `breccia`

`breccia` will port the exact architectural wins of **Infernix**, **Basalt**, and **Strata** into a vendor-agnostic **Rust + Vulkan** runtime tailored for multi-GPU workstations with constrained host RAM.



Key Engine Subsystems

1. **The Split-Residency Model Loader (Zero-Conversion GGUF + Artifact Cache):**
   - Parses standard `IQ3_S` / `UD-Q4_K_XL` GGUF files directly via `memmap2` without requiring a 100 GB offline conversion step.
   - **Permanent Dense Pinning:** Extracts all non-expert tensors (attention projections, Gated DeltaNet state layers, routers, norms, and shared experts) and pins them permanently across VRAM (~5–6 GB total).
   - **Tiered Expert Pool:** With ~30 GB of system DDR5 RAM, you cannot hold all 57 GB of `IQ3_S` experts in RAM simultaneously. `breccia` adopts Infernix's **SSD Expert Tier**:
     - **VRAM Expert Cache (~18–20 GB combined across both GPUs):** Holds the ~10,000–12,000 most frequently routed experts in device-local VRAM.
     - **Host Pinned Ring (~12 GB DDR5):** Holds warm experts in `VK_EXT_external_memory_host` zero-copy buffers.
     - **NVMe Direct Tier (`tokio-uring`):** Streams cold expert misses and 4 KiB PLE `.ngram` blocks directly from NVMe with `O_DIRECT`, preventing OS page-cache thrashing and out-of-memory crashes.
2. **Asymmetric Dual-GPU Vulkan Scheduler (`breccia-vk`):**
   - Rather than splitting layers 0–16 on GPU 0 and 17–32 on GPU 1, `breccia` splits **by operational intensity** when configured or auto-detected:
     - **Compute/Dense Node (RX 6950 XT):** Runs the high-clock dense attention layers, router projections, and the **DFlash2 / MTP speculative drafter**. While it computes the router logits for Layer $L + 1$, it fires a Vulkan timeline semaphore alerting the DMA engine which experts are needed next.
     - **Bandwidth/Expert Node (MI50):** Uses its 1 TB/s HBM2 bandwidth to hold the massive LRU expert pool and execute the memory-bound `IQ3_S` expert matrix-vector multiplications and PagedAttention KV lookups.
3. **Vulkan Kernels & Optimizations Not Being Done Today:**
   - **GSQ-RCO & `IQ3_S` Register-Tile Dequantizers:** Custom SPIR-V shaders using non-uniform subgroup operations (`VK_EXT_subgroup_size_control`) and importance-grid lookup tables stored in Vulkan **Shared Memory (LDS)** rather than global VRAM buffers.
   - **Route-Tail Skipping (from Strata):** In MoE routing, if the lowest-weighted top-k expert has a router probability below a dynamic threshold (ϵ), `breccia` skips fetching it if it isn't currently cached in VRAM, eliminating PCIe/NVMe stalls at near-zero perplexity cost.
   - **Fused PLE + RMSNorm + QKV Kernels:** Eliminates intermediate VRAM writes between embedding lookups and attention projections.
4. **Hybrid Context & Cold KV System:**
   - **Radix-Tree Prefix Persistence:** Saves prefix blocks to disk (like Infernix's `--prefix-cache-save-mins`) so multi-agent coding sessions keep warm context across restarts.
   - **BeeLlama Tail + RotorQuant Body:** Keeps the last 256 tokens of each agent slot in uncompressed FP16, while rotating older KV blocks via 4D Quaternion Givens rotations (`RotorQuant`) into 3-bit/4-bit pages, allowing 4 to 6 concurrent parallel agents at 32K–64K context within your VRAM envelope.

3. Workspace & Repository Structure

Build `breccia` as a modular Rust Cargo workspace so individual components can be benchmarked and tested independently:

```
breccia/
├── Cargo.toml
├── crates/
│   ├── breccia-cli/        # CLI entry (`breccia serve`, `breccia profile`, `breccia bench`)
│   ├── breccia-profile/    # Hardware probing, micro-benchmarks, hw_profile.json solver
│   ├── breccia-vk/         # `ash` + `gpu-allocator` wrapper, timeline semaphores, DMA rings
│   ├── breccia-kernels/    # Slang/GLSL -> SPIR-V build pipeline (Wave32 & Wave64 targets)
│   ├── breccia-format/     # Zero-copy GGUF parser, .ngram splitter, VTC tensor indexing
│   ├── breccia-cache/      # LRU Expert VRAM Manager, Tier 2/3 KV Pager, RotorQuant, io_uring
│   ├── breccia-draft/      # DFlash2, MTP, and N-Gram speculative tree verifier
│   └── breccia-server/     # Continuous batching agent loop, OpenAI/Anthropic API, /v1/decide
└── shaders/
    ├── dequant_iq3s.slang  # LDS codebook lookup + subgroup dot product
    ├── paged_attn.slang    # Modular attention (Causal, Sliding Window, GDN, RotorQuant)
    └── moe_fused.slang     # Fused routed expert execution + SwiGLU
```

4. Phased Execution Plan (Local Target: 20 TPS → 85+ TPS)

Milestone	Core Deliverables	Target Outcome
M1: Prober & Async DMA Harness (Weeks 1–2)	• `breccia-profile` CLI (`--reprofile`, `--dry-run-plan`). • Vulkan dual-device initialization (`gfx906` Wave64 + `gfx1030` Wave32). • Async transfer queue benchmark (`VK_QUEUE_TRANSFER_BIT`) + `tokio-uring` 4 KiB NVMe reader.	Validated hardware JSON profile; proven saturated PCIe & NVMe async streaming with zero compute stalls.
M2: Split-Residency Loader & SPIR-V Kernels (Weeks 3–4)	• Zero-copy GGUF loader separating dense weights, `IQ3_S` routed experts, and `.ngram` PLE tables. • SPIR-V kernels for `IQ3_S` / `Q8_0` / `IQ4_NL` using LDS lookup tables. • Support for Qwen 3.8 Gated DeltaNet (GDN) linear layers and full QSA layers.	Single-token forward pass of `Qwen3.8-Flash-Next` executing cleanly across both AMD GPUs.
M3: LRU Expert Cache & Route Prefetching (Weeks 5–6)	• VRAM slot allocator holding ~11,000 hot experts across MI50 + RX 6950 XT. • Layer $L+1$ route predictor + Strata-style route-tail skipping. • Fallback `io_uring` SSD expert tier for cache misses beyond your 30 GB host RAM.	Elimination of CPU layer fallbacks; baseline single-stream speed jumps from **20 TPS to ~45–55 TPS**.
M4: Speculative Drafting (DFlash2 + MTP) (Weeks 7–8)	• Integration of native MTP draft heads + Inco AI's **DFlash2** (776 MB) drafter on the RX 6950 XT. • N-gram copy-drafter for coding/agentic workflows. • Batched tree verification pass on the MI50.	2.2x–3.0x token acceptance multiplier; effective decode speed hits **85–130+ TPS** on coding/structured output.
M5: Tiered KV & Parallel Agent Server (Weeks 9–10)	• PagedAttention with FP16 tail + 3-bit/4-bit RotorQuant body. • Radix-tree prefix cache with disk persistence.	**4 to 6 parallel agents** running concurrently without VRAM OOM or expert cache thrashing.

Milestone	Core Deliverables	Target Outcome
	• Agent state machine (auto-evicting `WaitingForTool` contexts to NVMe) + `/v1/chat/completions` & `/v1/decide`.	
M6: Tailnet Mesh Extension (Future Phase)	• `quinn` QUIC transport, `NodeManifest` gossip, and Tier 4 remote memory leases.	Seamless multi-machine scaling built on top of the local engine.

5. Immediate Next Step: Bootstrapping Milestone 1

To get started right away, we should scaffold the Cargo workspace and build **Milestone 1 (`breccia-profile` + `breccia-vk`)**:

1. Initialize the Vulkan instance and enumerate both your **MI50** and **RX 6950 XT**.
2. Query subgroup properties (`Wave64` vs `Wave32`), bar memory heaps, and Dedicated Transfer Queue indices.
3. Run the 2-second calibration sweep (VRAM GB/s, FP16/INT8 TFLOPS, PCIe DMA host-to-device GB/s, and `io_uring` 4 KiB random read IOPS) and emit the `hw_profile.json` partitioning plan for your `Qwen3.8-Flash-Next` GGUF.

Ready to generate the `Cargo.toml` workspace layout and the initial Rust code for `breccia-profile` and `breccia-vk` device discovery?
